

Deep16 (深十六) Architecture Specification Milestone 3

16-bit RISC Processor with Enhanced Memory Addressing

• • •

1. Processor Overview

Deep16 is a 16-bit RISC processor optimized for efficiency and simplicity:

- **16-bit fixed-length instructions**
- **16 general-purpose registers**
- **Segmented memory addressing** (1MB physical address space)
- **4 segment registers** for code, data, stack and extra
- **shadow register views** for interrupts
- **Hardware-assisted interrupt handling**
- **Complete word-based memory system** (no byte operations)
- **Extended addressing** 20-bit physical address space
- **5-stage pipelined implementation** with delayed branch
- **Non-forwarded load capability** for architectural state access

1.1 Key Features

- All instructions exactly 16 bits
- 16 user-visible registers, PC is R15
- 4 segment registers: CS, DS, SS, ES
- Processor status word (PSW) for flags and implicit segment selection
- PC'/CS'/PSW' shadow views for interrupt handling
- Compact encoding with variable-length opcodes
- Enhanced memory addressing with stack/extra registers
- **1-slot delayed branch** for improved pipeline efficiency
- **Word-only memory operations** (simplifies alignment)
- **No memory protection** (keep it simple)
- **Universal MOV instruction** with automatic encoding selection
- **Enhanced assembler syntax** with bracket and plus notation
- **Architectural register access** via MOV with immediate=3

• • •

2. Register Set

2.1 General Purpose Registers (16-bit)

Table A: General Purpose Registers

Register	Alias	Conventional Use
R0		LDI destination, temporary
R1-R11		General purpose
R12	FP	Frame Pointer

Opcode	Bits	Instruction	Format	Pipeline Notes
110	3	ALU2	[110][func5][Rd4][Rs/imm4]	Full pipeline, forwarding
1110	4	JMP	[1110][type3][target9]	Uses delay slot
11110	5	LDS/STS	[11110][d1][seg2][Rd4][Rs4]	Segment access in MEM
111110	6	MOV	[111110][Rd4][Rs4][imm2]	imm2=3 disables forwarding
1111110	7	LSI	[1111110][Rd4][imm5]	Full pipeline
11111110	8	SOP	[11111110][type4][Rx/imm4]	Various pipeline effects
111111110	9	MVS	[111111110][d1][Rd4][seg2]	Segment access in MEM
1111111110	10	SMV	[1111111110][src2][Rd4]	Special register access
111111111110	13	SYS	[111111111110][op3]	Pipeline flush on RETI
11111111111111	16	HLT	[11111111111111]	Halt the processor

3.2 Data Movement Instructions

Table E: Data Movement Instructions

Instruction	Format	Binary Encoding	Behavior
MOV	MOV Rd, Rs, imm	111110 Rd4 Rs4 imm2	Rd = Rs + imm
LDI	LDI imm	0 imm15	R0 = imm
LSI	LSI Rd, imm	1111110 Rd4 imm5	Rd = imm (sign-extended)
MVS	MVS Rd, Sx	111111110 0 Rd4 seg2	Rd = Sx
MVS	MVS Sx, Rd	111111110 1 Rd4 seg2	Sx = Rd
SMV	SMV Rd, APC	1111111110 00 Rd4	Rd = PC'
SMV	SMV Rd, APSW	1111111110 01 Rd4	Rd = PSW'
SMV	SMV Rd, PSW	1111111110 10 Rd4	Rd = PSW
SMV	SMV Rd, ACS	1111111110 11 Rd4	Rd = CS'

MOV with immediate value 3 (AMV/ALNK aliases):

- Bypasses all pipeline forwarding mechanisms
- Reads the current architectural state from register file
- Essential for reading PC in branch delay slots
- Causes 1-cycle stall since it cannot use forwarded values
- Provides synchronization point for reading stable state

3.3 ALU Instructions - Group 1: Basic Operations

Table F: Basic ALU Instructions

Instruction	Format	Binary Encoding	Behavior
ADD	ADD Rd, Rs	110 00000 Rd4 Rs4	Rd = Rd + Rs, set flags
ADD	ADD Rd, imm	110 00001 Rd4 imm4	Rd = Rd + imm, set flags
SUB	SUB Rd, Rs	110 00010 Rd4 Rs4	Rd = Rd - Rs, set flags
SUB	SUB Rd, imm	110 00011 Rd4 imm4	Rd = Rd - imm, set flags
CMP	CMP Rd, Rs	110 00100 Rd4 Rs4	Rd - Rs, set flags only
CMP	CMP Rd, imm	110 00101 Rd4 imm4	Rd - imm, set flags only
AND	AND Rd, Rs	110 00110 Rd4 Rs4	Rd = Rd & Rs, set flags
AND	AND Rd, imm	110 00111 Rd4 imm4	Rd = Rd & imm, set flags
TBC	TBC Rd, Rs	110 01000 Rd4 Rs4	Rd & Rs, set flags only
TBC	TBC Rd, count	110 01001 Rd4 count4	Rd & (1<<count), Z=1 if bit CLEAR
OR	OR Rd, Rs	110 01010 Rd4 Rs4	`Rd = Rd
OR	OR Rd, imm	110 01011 Rd4 imm4	`Rd = Rd

Instruction	Format	Binary Encoding	Behavior
XOR	XOR Rd, Rs	110 01100 Rd4 Rs4	$Rd = Rd \wedge Rs$, set flags
XOR	XOR Rd, imm	110 01101 Rd4 imm4	$Rd = Rd \wedge imm$, set flags
TBS	TBS Rd, Rs	110 01110 Rd4 Rs4	$Rd \wedge Rs$, set flags only
TBS	TBS Rd, count	110 01111 Rd4 count4	$Rd \wedge (1 \ll count)$, Z=1 if bit SET

3.4 ALU Instructions - Group 2: Shift/Rotate Operations

Table G: Shift and Rotate Instructions

Instruction	Format	Binary Encoding	Behavior
SL	SL Rd, count	110 10000 Rd4 count4	$Rd = Rd \ll count$, C = MSB
SLA	SLA Rd, count	110 10001 Rd4 count4	$Rd = Rd \ll count$ (arithmetic, preserves sign)
SLAC	SLAC Rd, count	110 10010 Rd4 count4	$Rd = (Rd \ll count) \mid (C \ll (count-1))$ (arithmetic)
SLC	SLC Rd, count	110 10011 Rd4 count4	$Rd = (Rd \ll count) \mid (C \ll (count-1))$, C = MSB
SR	SR Rd, count	110 10100 Rd4 count4	$Rd = Rd \gg count$, C = LSB
SRC	SRC Rd, count	110 10101 Rd4 count4	$Rd = (Rd \gg count) \mid (C \ll (15-count))$, C = LSB
SRA	SRA Rd, count	110 10110 Rd4 count4	$Rd = Rd \gg count$ (arithmetic), C = LSB
SRAC	SRAC Rd, count	110 10111 Rd4 count4	$Rd = (Rd \gg count) \mid (C \ll (15-count))$ (arithmetic)
ROL	ROL Rd, count	110 11000 Rd4 count4	$Rd = (Rd \ll count) \mid (Rd \gg (16-count))$
RLC	RLC Rd, count	110 11001 Rd4 count4	$Rd = (Rd \ll count) \mid (C \ll (count-1)) \mid (Rd \gg (16-count))$, C = bit shifted out
ROR	ROR Rd, count	110 11010 Rd4 count4	$Rd = (Rd \gg count) \mid (Rd \ll (16-count))$
RRC	RRC Rd, count	110 11011 Rd4 count4	$Rd = (Rd \gg count) \mid (C \ll (15-count)) \mid (Rd \ll (16-count))$, C = bit shifted out

Note: Arithmetic shifts (SLA, SLAC, SRA, SRAC) preserve the sign bit for two's complement operations, while logical shifts (SL, SLC, SR, SRC) treat values as unsigned.

RLC/RRC Carry Behavior:

- **RLC:** Carry bit is inserted at the LSB position, bit shifted out becomes new carry
- **RRC:** Carry bit is inserted at the MSB position, bit shifted out becomes new carry
- **Purpose:** Enables multi-word rotations by propagating carry between words

3.5 ALU Instructions - Group 3: Multiply/Divide Operations

Table H: Multiply/Divide Instructions

Instruction	Format	Binary Encoding	Behavior
MUL	MUL Rd, Rs	110 11100 Rd4 Rs4	$Rd = Rd * Rs$ ($16 \times 16 \rightarrow 16$ -bit)
MUL32	MUL32 Rd, Rs	110 11101 Rd4 Rs4	$R[d]:R[d+1] = Rd * Rs$ (Rd must be UNEVEN)
DIV	DIV Rd, Rs	110 11110 Rd4 Rs4	$Rd = Rd / Rs$ ($16 \div 16 \rightarrow 16$ -bit quotient)
DIV32	DIV32 Rd, Rs	110 11111 Rd4 Rs4	$R[d]:R[d+1] = Rd / Rs$ (Rd must be UNEVEN)

32-bit Operation Requirements:

- MUL32/DIV32 require UNEVEN register numbers (1,3,5,7,9,11,13)
- Assembler must enforce this constraint
- Runtime behavior is undefined if even register specified
- Result: $R[d] = \text{low 16 bits}$, $R[d+1] = \text{high 16 bits}$
- For MUL32: $R[d]:R[d+1] = Rd \times Rs$
- For DIV32: $R[d] = \text{quotient}$, $R[d+1] = \text{remainder}$

3.6 Single Operand ALU Operations

Table I: Single Operand Instructions

Instruction	Format	Binary Encoding	Behavior
SWB	SWB Rx	11111110 0000 Rx4	$Rx = (Rx \ll 8)$
INV	INV Rx	11111110 0001 Rx4	$Rx = \sim Rx$
NEG	NEG Rx	11111110 0010 Rx4	$Rx = -Rx$

3.7 Memory Access Instructions

Table J: Memory Access Instructions

Instruction	Format	Binary Encoding	Behavior
LD	LD Rd, Rb, offset	10 0 Rd4 Rb4 offset5	$Rd = \text{Mem}[\text{DS}:(Rb + \text{offset})]$
ST	ST Rd, Rb, offset	10 1 Rd4 Rb4 offset5	$\text{Mem}[\text{DS}:(Rb + \text{offset})] = Rd$
LDS	LDS Rd, seg, Rb	11110 0 seg2 Rd4 Rb4	$Rd = \text{Mem}[\text{seg}:Rb]$
STS	STS Rd, seg, Rb	11110 1 seg2 Rd4 Rb4	$\text{Mem}[\text{seg}:Rb] = Rd$

3.8 Control Flow Instructions

Table K: Condition Codes for Jump Instructions

Condition	Code	Mnemonic	Test
Zero	000	JZ	$Z = 1$
Not Zero	001	JNZ	$Z = 0$
Carry	010	JC	$C = 1$
No Carry	011	JNC	$C = 0$
Negative	100	JN	$N = 1$
Not Negative	101	JNN	$N = 0$
Overflow	110	JO	$V = 1$
No Overflow	111	JNO	$V = 0$

Table L: Control Flow Instructions

Instruction	Format	Binary Encoding	Behavior
JZ	JZ target	1110 000 target9	if (Z) $PC = PC + 1 + \text{target}$
JNZ	JNZ target	1110 001 target9	if (!Z) $PC = PC + 1 + \text{target}$
JC	JC target	1110 010 target9	if (C) $PC = PC + 1 + \text{target}$
JNC	JNC target	1110 011 target9	if (!C) $PC = PC + 1 + \text{target}$
JN	JN target	1110 100 target9	if (N) $PC = PC + 1 + \text{target}$
JNN	JNN target	1110 101 target9	if (!N) $PC = PC + 1 + \text{target}$
JO	JO target	1110 110 target9	if (V) $PC = PC + 1 + \text{target}$
JNO	JNO target	1110 111 target9	if (!V) $PC = PC + 1 + \text{target}$
JML	JML Rx	11111110 0100 Rx4	$CS = R[Rx], PC = R[Rx+1]$

3.9 PSW Operations

Table M: PSW Segment Assignment Operations

Instruction	Format	Binary Encoding	Behavior
SRS	SRS Rx	11111110 1000 Rx4	$PSW.SR = Rx, PSW.DS = 0$
SRD	SRD Rx	11111110 1001 Rx4	$PSW.SR = Rx, PSW.DS = 1$
ERS	ERS Rx	11111110 1010 Rx4	$PSW.ER = Rx, PSW.DE = 0$
ERD	ERD Rx	11111110 1011 Rx4	$PSW.ER = Rx, PSW.DE = 1$

Table N: PSW Flag Operations

Instruction	Format	Binary Encoding	Behavior
SET	SET imm	11111110 1100 imm4	PSW[imm] = 1
CLR	CLR imm	11111110 1101 imm4	PSW[imm] = 0
SET2	SET2 imm	11111110 1110 imm4	PSW[imm+4] = 1
CLR2	CLR2 imm	11111110 1111 imm4	PSW[imm+4] = 0

3.10 System Operations

Table O: System Instructions

Instruction	Format	Binary Encoding	Behavior
NOP	NOP	1111111111110 000	No operation
FSH	FSH	1111111111110 001	Flush pipeline
SWI	SWI	1111111111110 010	Software interrupt
RETI	RETI	1111111111110 011	Return from interrupt

3.11 Halt Instruction

Table P: Halt Instruction

Instruction	Format	Binary Encoding	Behavior
HLT	HLT	1111111111111111	Halt processor

● ● ●

4. Enhanced Assembler Syntax and Aliases

4.1 Enhanced Assembler Syntax (Preprocessing Only)

Important: The enhanced syntax described below is purely **assembler preprocessing**. The binary encoding always uses the specific instruction (MOV, MVS, SMV, LD, ST). The assembler automatically translates enhanced syntax to the correct machine instruction.

4.1.1 LD/ST Bracket Syntax

Assembler Input (Enhanced Syntax):

LD	R1, [R2+5]	; Assembler preprocessing
ST	R1, [SP-4]	; Assembler preprocessing
LD	R1, [R2]	; Offset 0 implied

Actual Binary Encoding:

LD	R1, R2, 5	; Machine instruction: [10][0][R1][R2][5]
ST	R1, SP, 4	; Machine instruction: [10][1][R1][SP][4]
LD	R1, R2, 0	; Machine instruction: [10][0][R1][R2][0]

4.1.2 MOV Plus Syntax

Assembler Input (Enhanced Syntax):

MOV	R1, R2+3	; Assembler preprocessing
MOV	R3, SP-4	; Assembler preprocessing

Actual Binary Encoding:

MOV	R1, R2, 3	; Machine instruction: [111110][R1][R2][3]
MOV	R3, SP, 0	; Note: Negative offsets not supported in MOV

4.2 Instruction Aliases

Table Q: Instruction Aliases

Alias	Actual Instruction	Purpose
HALT	HLT	Halt processor
JMP Rx	MOV PC, Rx	Unconditional jump to register
LNK Rx	MOV Rx, PC, 2	Link to subroutine (standard case)
LINK	MOV LR, PC, 2	Link to subroutine using LR (standard case)
AMV Rx, Ry	MOV Rx, Ry, 3	Architectural move (bypass forwarding)
ALNK Rx	MOV Rx, PC, 3	Architectural link in delay slot
ALINK	MOV LR, PC, 3	Architectural link to LR in delay slot

4.3 Flag Operation Aliases

Table R: Common Flag Aliases

Alias	Actual Instruction	Purpose
SETN	SET 0	Set Negative flag
CLRN	CLR 0	Clear Negative flag
SETZ	SET 1	Set Zero flag
CLRZ	CLR 1	Clear Zero flag
SETV	SET 2	Set Overflow flag
CLRV	CLR 2	Clear Overflow flag
SETC	SET 3	Set Carry flag
CLRC	CLR 3	Clear Carry flag
SETI	SET2 0	Enable interrupts
CLRI	CLR2 0	Disable interrupts
SETS	SET2 1	Enable shadow view
CLRS	CLR2 1	Disable shadow view

4.4 Bit Test Usage Examples

Natural bit testing with corrected semantics:

```
; Test if bit 3 is SET - jump if it IS set
TBS    R5, 3      ; Z=0 if bit 3 is SET, Z=1 if CLEAR
JNZ    bit_is_set ; Jump when Z=0 (bit WAS set)

; Test if bit 7 is CLEAR - jump if it IS clear
TBC    R5, 7      ; Z=0 if bit 7 is CLEAR, Z=1 if SET
JNZ    bit_is_clear ; Jump when Z=0 (bit WAS clear)

; Alternative: test if NOT set
TBS    R5, 3
JZ     bit_not_set ; Jump when Z=1 (bit was NOT set)
```

4.5 MOV Special Immediate Value

The immediate value 3 in MOV instructions has special meaning:

- **MOV Rx, Ry, 3:** Architectural register read - reads the current architectural value of Ry, ignoring any pending writes in the pipeline
- When detected by hardware (immediate value = 3), forwarding is bypassed and the register file is read directly
- This enables correct PC reading for link instructions in delay slots
- For general registers, this provides a mechanism to read stable architectural state
- **Pipeline effect:** Causes 1-cycle stall since it cannot use forwarded values

5. Interrupt System

5.1 Interrupt Vector Table

Located at Segment 0 (Low Memory):

```
0x0000: RESET_VECTOR      (PC loaded from here on reset)
0x0001: HW_INT_VECTOR     (PC loaded from here on hardware interrupt)
0x0002: SWI_VECTOR       (PC loaded from here on software interrupt)
```

5.2 Reset State

- **Initial registers:** CS = 0xFFFF, DS = 0x1000, SS = 0x8000, ES = 0x2000, SP (R13) = 0x7FFF, PC (R15) = 0x0000, PSW = 0x0000
- **Boot ROM** at 0xFFFF0..0xFFFFF executes first and establishes runtime segments, performs basic diagnostics, and jumps to low memory.
- **Execution begins** at physical address computed by the boot ROM's jump (default CS:PC = 0x0000:0x0100).

5.2.1 Boot ROM Sequence (at 0xFFFF0)

```
0xFFFF0: 0x0000      ; LDI  #0x0000 -> R0
0xFFFF1: 0xFF41      ; MVS  DS, R0
0xFFFF2: 0xFF42      ; MVS  SS, R0
0xFFFF3: 0xFC21      ; LSI  R1, 1
0xFFFF4: 0xFE01      ; SWB  R1          ; R1 = 0x0100 (user program start address)
0xFFFF5: 0xA200      ; ST   R1, [R0+0]
0xFFFF6: 0xA201      ; ST   R1, [R0+1]
0xFFFF7: 0xA202      ; ST   R1, [R0+2]
0xFFFF8: 0xFE40      ; JML  R0          ; Jump to CS=R0, PC=R1 (0x0000:0x0100)
0xFFFF9: 0xFFFF0     ; NOP              ; Delay slot
0xFFFFA: 0xFFFF      ; HLT
0xFFFFB: 0xFFFF      ; HLT
0xFFFFC: 0xFFFF      ; HLT
0xFFFFD: 0xFFFF      ; HLT
0xFFFFE: 0xFFFF      ; HLT
0xFFFFF: 0xFFFF      ; HLT
```

Default effect:

- Sets DS = 0x0000 and SS = 0x0000
- Prepares R1 = 0x0100 via LSI and SWB
- Stores diagnostic words at physical 0x000000..
- Performs JML R0 using the (R0,R1) pair → jumps to CS=0x0000, PC=0x0100

5.3 Shadow Register System

On Interrupt:

- PSW' ← PSW (Snapshot pre-interrupt state)
- PSW'.S ← 1, PSW'.I ← 0 (Configure shadow context)
- CS ← 0 (Interrupts run in Segment 0)
- PC ← Mem[interrupt_vector] (Load PC from vector table)
- **Pipeline flushed** to ensure clean context switch

On RETI:

- Switch to normal view (PSW.S=0)
- No register copying - pure view switching
- Both contexts preserved for debugging
- **Pipeline flushed** on context restoration

6. Memory Architecture

6.1 Physical Memory Map (1MB)

```
0x00000 - 0xDFFFF: Home Segment (896KB) - Code execution starts here
0xE0000 - 0xFFFFF: Graphics Segment (64KB) - Reserved for future 640x400 display
0xF0000 - 0xFFFFF: I/O Segment (64KB) - Memory-mapped peripherals
    └─ 0xF1000 - 0xF17CF: Screen Buffer (2KB) - 80x25 character display
0xFFFF0 - 0xFFFFF: Boot ROM (16 words) - Initial boot sequence
```

6.2 Memory Access Characteristics

- **Word-based only** - No byte addressable operations
- **No alignment restrictions** - All addresses are word-aligned
- **No memory protection** - Simple and predictable
- **Memory-mapped I/O** - Peripherals accessed via load/store

6.3 Screen Memory Mapping

- **Location:** 0xF1000-0xF17CF (80x25 characters × 2 bytes)
- **Format:** Lower byte = ASCII character, Upper byte = attributes (reserved)
- **Access:** Use ES segment with offset for efficient writes

Correct screen setup:

```
LDI  0x0FFF      ; LDI always loads R0
INV  R0          ; R0 = 0xF000
MVS  ES, R0      ; ES = 0xF000
LDI  0x0000      ; Base offset to R0
MOV  R10, R0     ; Copy to R10
ERD  R10         ; Use R10/R11 for ES access, sets DE=1 automatically
```

● ● ●

7. Pipeline Architecture

7.1 5-Stage Pipeline Structure

Pipeline Stages:

1. **IF** (Instruction Fetch) - Fetch instruction from memory using CS:PC
2. **ID** (Instruction Decode) - Decode instruction, read registers, resolve hazards
3. **EX** (Execute) - ALU operations, effective address calculation, branch resolution
4. **MEM** (Memory Access) - Load/store operations, segment register access
5. **WB** (Write Back) - Write results to register file

7.2 Delayed Branch Implementation

One delay slot is implemented for all branch and jump instructions:

- The instruction immediately following a branch/jump **always executes**
- Compiler/assembler must schedule useful instructions in the delay slot
- **Applies to:** All conditional jumps (JZ, JNZ, JC, JNC, JN, JNN, JO, JNO), JML

Example Optimization:

```
; Traditional approach (wasted cycle)
LNK  R14          ; MOV R14, PC, 2
JMP  subroutine
NOP                      ; Wasted delay slot
return_here:           ; Execution continues here

; Optimized approach (uses delay slot)
JMP  subroutine
```

```

ALNK R14          ; MOV R14, PC, 3 (architectural read)
return_here:      ; Execution continues here

; Both approaches set R14 to 'return_here'

; Suboptimal - empty delay slot
ADD R1, R2
JZ  target
NOP                ; Wasted cycle

; Optimized - useful work in delay slot
ADD R1, R2
JZ  target
MOV R3, R4        ; Useful work executes regardless of branch

```

7.3 Forwarding and Hazard Handling

The pipeline implements comprehensive forwarding to resolve data hazards:

Standard Forwarding:

- EX/MEM → EX: Forward results from previous ALU operation
- MEM/WB → EX: Forward results from load/memory operations
- Eliminates most data hazard stalls

Architectural Register Access:

- **MOV Rx, Ry, 3:** Bypasses all forwarding mechanisms
- Reads the current architectural value from register file
- Essential for correct PC reading in delay slot link instructions
- Provides stable state access for synchronization operations

Pipeline Flush Behavior: Pipeline flushes occur on:

- RETI instruction (context switch)
- Interrupt entry
- JML instruction (far jump) Flushes clear the pipeline and refetch from new context.

7.4 Performance Characteristics

- **Base CPI:** Ideally 1.0 (one instruction per cycle)
- **Realistic CPI:** 1.1-1.3 due to stalls and multi-cycle operations
- **Branch penalty:** 0 cycles (thanks to delayed branch)
- **Load-use penalty:** 1 cycle stall when unavoidable
- **FPGA Target:** 80MHz achievable in modern FPGAs



8. Programming Model

8.1 Register Usage Conventions

Register	Preserved?	Purpose
R0	Caller-save	LDI destination, temporary
R1-R11	Caller-save	General purpose
R12 (FP)	Callee-save	Frame pointer
R13 (SP)	Callee-save	Stack pointer
R14 (LR)	Callee-save	Return address
R15 (PC)	-	Program counter

8.2 Stack Frame Layout

```
High addresses
+-----+
| Saved LR   | ← FP + 3
+-----+
| Saved FP   | ← FP + 2
+-----+
| Local 2    | ← FP + 1
+-----+
| Local 1    | ← FP
+-----+
| Parameter n| ← FP - 1
+-----+
| ...        |
+-----+
| Parameter 1| ← FP - n + 1
+-----+
Low addresses
```

8.3 Common Idioms

Function Prologue (Enhanced Syntax):

```
; Save frame and link, allocate stack space
MOV  FP, SP          ; Set new frame pointer
LSI  R0, -4          ; Allocate 4 words to R0
ADD  SP, SP, R0       ; Adjust stack pointer
ST   LR, [FP+3]      ; Save return address using bracket syntax
ST   OldFP, [FP+2]    ; Save old frame pointer
```

Correct Screen Output (Enhanced Syntax):

```
; Efficient screen writing using ES segment
setup_screen:
    LDI  0x0FFF        ; LDI always loads R0
    INV  R0            ; R0 = 0xF000
    MOV  ES, R0        ; Enhanced MOV to segment register
    LDI  0x0000        ; Base offset to R0
    MOV  R10, R0       ; Enhanced MOV syntax
    ERD  R10           ; Use R10/R11 for ES access, sets DE=1

write_char:
    LDI  'H'          ; Character to R0
    MOV  R1, R0        ; Copy to R1
    STS  R1, [R10+0x1000] ; Enhanced bracket syntax for screen write

    RET
```

Multi-Word Rotation Example:

```
; Multi-word 64-bit rotate left using RLC
; Assume R1:R2:R3:R4 contains 64-bit value
RLC  R1, 1            ; Rotate low word left through carry
RLC  R2, 1            ; Rotate next word, carry propagates
RLC  R3, 1            ; Continue propagation
RLC  R4, 1            ; Rotate high word

; Multi-word 64-bit rotate right using RRC
RRC  R4, 1            ; Rotate high word right through carry
RRC  R3, 1            ; Rotate next word, carry propagates
RRC  R2, 1            ; Continue propagation
RRC  R1, 1            ; Rotate low word
```

Far Function Call:

```
far_call:
    ; R2 = target segment, R3 = target offset
    ST  R2, [SP-1]    ; Save target segment
    ST  R3, [SP-2]    ; Save target offset
    LDI return_here
    MOV R4, R0        ; Copy to R4
    ST  R4, [SP-3]    ; Save return address
```

```

JML R2          ; Far jump to CS=R2, PC=R3

return_here:
; Execution continues here after far return

```

Interrupt Handler:

```

interrupt_handler:
; Automatic context switch to shadow registers
SMV R0, APSW      ; Read pre-interrupt PSW to R0
MOV R5, R0        ; Copy to R5 for processing
SMV R0, APC       ; Read pre-interrupt PC to R0
MOV R6, R0

; Interrupt processing...

RETI              ; Return and restore context

```

8.4 Segment Register Conventions

Stack Segment Access:

- **Option A:** PSW.SR = 13 (SP), PSW.DS = 0 → SP alone accesses SS
- **Option B:** PSW.SR = 12 (FP), PSW.DS = 1 → FP/SP pair accesses SS

Extra Segment (Screen) Access:

- ES = 0xF000, PSW.ER = 11, PSW.DE = 0 → R11 alone accesses screen
- ES = 0xF000, PSW.ER = 10, PSW.DE = 1 → R10/R11 pair accesses screen

Example screen setup:

```

LDI 0x0FFF      ; R0 = 0x0FFF
INV R0          ; R0 = 0xF000
MVS ES, R0      ; ES = 0xF000
LSI R10, 0      ; R10 = 0x0000
ERD R10         ; Use R10/R11 for ES access

```



9. Implementation Notes

9.1 FPGA Implementation

- **Target Frequency:** 80MHz in modern FPGAs
- **Memory Interface:** 20 address lines, 16 data lines
- **Block RAM:** Can be used for zero-wait-state memory
- **Pipeline registers:** Standard flip-flop implementation

9.2 Simulator Integration

- **DeepCode IDE:** Complete development environment
- **Assembler:** Supports all enhanced syntax features
- **Simulator:** Cycle-accurate with visual debugging
- **Screen Subsystem:** 80×25 character display at 0xF1000

9.3 Toolchain Support

- **Assembler Directives:** .code, .data, .org, .word, .text
- **Label Support:** Forward and backward references
- **Error Reporting:** Comprehensive with line numbers
- **Listing Output:** Includes addresses and generated code
- **Enhanced Syntax:** Bracket notation for LD/ST, plus notation for MOV
- **Instruction Aliases:** HALT, JMP, LNK, LINK, AMV, ALNK, ALINK, and flag operations

Key Updates:

- ☒ Complete ALU2 instruction redesign with 5-bit function field
- ☒ 32 function codes organized into 3 logical groups
- ☒ Corrected TBS/TBC semantics for intuitive bit testing
- ☒ Full shift/rotate instruction set with arithmetic variants
- ☒ Corrected RLC/RRC descriptions with proper carry bit handling
- ☒ UNEVEN register requirement for 32-bit multiply/divide
- ☒ Reorganized document structure (ISA first, then system details)
- ☒ Table numbering with letters (A-R) for clear reference
- ☒ Boot ROM sequence corrected with user program start comment
- ☒ Enhanced educational focus with clear examples
- ☒ All non-standard behaviors thoroughly explained
- ☒ Pipeline implications for MOV immediate=3 clarified
- ☒ Segment register conventions documented
- ☒ Complete bit test examples included
- ☒ Delayed branch examples expanded
- ☒ Multi-word rotation examples added

This revision represents a significant improvement in both encoding efficiency and educational clarity while maintaining the core RISC philosophy.